BioSync

Biometric Health Tracking Platform

Project Binder

CSC 289 — Programming Capstone

Spring 2026

Group 2

Sarah Ayres · Joshua Eckman · Myra Williams · Toni Wilson

Repository: https://github.com/CSC-289-Biometric-Group-2/BioSync-Group2

Date of Final Submission: _____

1. Project Overview

What Is BioSync?

BioSync is a biometric health tracking web application developed as the capstone project for CSC 289 — Programming Capstone at Wake Technical Community College, Spring 2026. The application consolidates health data collection, storage, and analysis into a single accessible platform, addressing a genuine gap in how patients and caregivers currently manage health information across disconnected systems.

The application serves two distinct user types: patients who wish to track, store, and manage their own biometric and medical data, and caregivers who require structured, secure access to the health information of the patients they support. Rather than specializing in a single health domain — as most fitness or medical apps do — BioSync is designed as a comprehensive hub that accommodates a broad range of health metrics including heart rate, blood pressure, blood oxygen level, and more, alongside uploaded medical documents, medication tracking, and provider-ready trend data.

The Problem BioSync Addresses

Health tracking today is fundamentally fragmented. A typical patient managing a chronic condition might maintain separate accounts across a fitness tracker, a patient portal for their primary care physician, a separate portal for a specialist, and a medication management app — none of which communicate with one another. When that patient visits a doctor, they often arrive underprepared, unable to produce a coherent summary of their health data across those platforms.

The problem becomes even more acute when hospitals are involved. When a patient is admitted to a local hospital, medications are commonly adjusted — doses changed, new prescriptions added, others discontinued. These changes can take days to propagate to a VA facility or a primary care physician. In the interim, no single party has a complete, current picture of the patient's medication status. This is not an edge case. It is a routine failure of our healthcare information infrastructure.

For caregivers, the situation is compounded further. Accessing a patient's health records often requires logging in with that patient's username and password — a practice that violates HIPAA guidelines and creates serious security risks, particularly for caregivers who support multiple patients across multiple systems. There is no standardized, secure mechanism for a caregiver to maintain a consolidated view of their patients without compromising those patients' account security or seeing more information than their role requires.

BioSync was designed specifically to solve both sides of this problem. Patients get a single hub for all their health data — uploaded records, logged readings, a living medication list, and trend data ready to share at any appointment. Caregivers get a secure, role-scoped view of their patients, granted through an explicit patient-initiated linking process, without ever needing to share credentials or access more than what is appropriate to their role.

Project Background and Inspiration

BioSync was developed in collaboration with the Graphic Design program at Wake Tech. GD students created a series of health-technology brand concepts under the shared prompt of biometric and health applications. Group 2 was assigned the VitalWatch concept — a connected heart health monitoring brand built around role-based access, a caregiver-patient separation model, and a clean medical color palette. BioSync drew on that visual system, extending it into a fully functional Flask web application while reinterpreting the concept around the team's own technical decisions and personal priorities.

The personal context behind this project is meaningful. The team lead, Sarah Ayres, is an active caregiver whose mother is a veteran. The daily reality of managing care across VA and local hospital systems — where medication changes take days to communicate, where the only path to records is borrowing a patient's login, and where no existing tool adequately supports the caregiver role — directly shaped every feature priority in BioSync. This is not a theoretical problem the team researched. It is a lived one.

Current State of the Application

As of the final submission, BioSync is in a functional, semi-production state. The following capabilities are fully implemented and operational:

- User registration with role selection (Patient or Caregiver)
- Secure login and logout with session management
- Role-based dashboards with scoped data visibility
- Patient profile management and unique code generation
- Caregiver-to-patient linking via unique patient codes
- Biometric data logging across multiple metric types
- Health alert generation for out-of-range readings
- Medical document upload and server-side storage
- Medication list management with manual update capability

Import/export of health data is partially implemented — document upload is functional, and structured export to portable formats is in progress. Data-driven recommendations based on trend patterns are also partially implemented; the alert threshold system works, and pattern analysis for proactive recommendations requires further development. Bluetooth and API device connectivity — allowing direct reading from home monitors — is planned and architecturally anticipated in the data model, but not yet built.

2. Team Roster & Roles

Team Members

Group 2 consisted of four members, each contributing to the design, development, and delivery of BioSync across three Agile sprints. Roles were partially fixed and partially emergent based on team capacity and sprint priorities.

Name	Primary Role(s)	GitHub	Main Contribution
Sarah Ayres	Product Owner, Scrum Master, Frontend Lead, Presenter	Sayres-dev	Project direction, frontend development, security architecture, presentation, and overall integration
Joshua Eckman	Developer	Eckmanj8966	Application initialization, routing foundation, and heart rate threshold implementation
Myra Williams	Scrum Master, Developer	mwill2214	Sprint facilitation and development contributions (to be completed by team member)
Toni Wilson	Developer	toniwilson	Development contributions (to be completed by team member)

Sprint Assignments

BioSync was developed across three two-week sprints. Scrum Master responsibilities were shared, with Sarah Ayres taking primary ownership of sprint planning and backlog management, and Myra Williams supporting facilitation during Sprints 2 and 3.

Sprint	Scrum Master	Primary Focus
Sprint 1	Sarah Ayres	Project scaffolding, database schema, user registration and login, basic session management
Sprint 2	Myra Williams	Role selection, caregiver linking, profile management, biometric data logging, alerts, document upload
Sprint 3	Sarah Ayres	Security hardening, medication list, UI polish, deployment preparation, documentation

GitHub Collaboration

The team used a shared GitHub repository under the organization CSC-289-Biometric-Group-2. Development followed a feature-branch workflow — each team member worked in their own branch and submitted pull requests for review before merging to main. The Codespaces environment ensured that all members could develop and test against a consistent environment regardless of their local machine setup.

Repository: https://github.com/CSC-289-Biometric-Group-2/BioSync-Group2

3. Repository & Submission Details

Repository Information

All source code, documentation, and project assets for BioSync are maintained in the team's GitHub repository. The repository is organized to support collaborative development, code review, and deployment.

- Repository URL: https://github.com/CSC-289-Biometric-Group-2/BioSync-Group2
- Organization: CSC-289-Biometric-Group-2
- Primary branch: main — contains the stable, submission-ready codebase
- Development branches: feature branches per team member, merged via pull request
- GitHub Codespaces: used as the primary development environment for all team members

Repository Structure

The repository follows a standard Flask application layout with separation between application logic, templates, static assets, and data storage:

```
BioSync-Group2/   app.py                Application entry point and route
definitions   schema.sql             Database schema initialization script
requirements.txt       Python dependency list   README.md              Project
overview and setup instructions   templates/             Jinja2 HTML templates
base.html              Base layout template with navigation     auth/
Login and registration templates     dashboard/             Patient and caregiver
dashboard templates     profile/               Profile view and edit templates
biometrics/            Reading log and alert templates   static/     style.css
Application stylesheet     images/                Logo and UI image assets
uploads/               Server-side document storage (user-scoped)   instance/
SQLite database file (gitignored in production)
```

Submission Details

- Course: CSC 289 — Programming Capstone
- Semester: Spring 2026
- Instructor: [Instructor name]
- Group: Group 2
- Date of final submission: _____
- Submitted deliverables: Source code repository, project binder, pitch presentation, demo application

4. Product Vision

4a. Target Users / Personas

BioSync was designed with three representative user types in mind. These personas were developed from real-world needs identified during the project concept phase and directly informed feature prioritization throughout development.

Persona 1 — Joy Edwards, Graduate Student

Joy Edwards is a 26-year-old graduate student studying nursing at a local university. She is health-conscious and motivated to track her wellness alongside her demanding academic schedule. Joy already uses a fitness tracker for step count and sleep monitoring, a separate app to log her manually-measured blood pressure, and her university's health portal for medical visit records. She finds the fragmentation exhausting — she can never see her full health picture in one view, and when she goes to a health appointment she inevitably forgets which app has which piece of information.

What Joy needs from BioSync: a single dashboard that pulls together all her tracked metrics, stores her visit summaries, and lets her quickly pull up trends to share with a provider. She is reasonably tech-savvy and comfortable with a web-based tool, but she doesn't want to spend time managing the system — she wants it to stay out of her way and surface information when she needs it. BioSync's health hub and document upload features are designed directly for users like Joy.

Would Joy come back? Yes — if uploading a new visit summary or logging a reading takes under 60 seconds and her dashboard immediately reflects the update. The friction of data entry is her biggest barrier; if BioSync makes that fast and obvious, she stays.

Persona 2 — Jason Spencer, Professional Caregiver

Jason Spencer is a 44-year-old licensed home health aide who currently supports three elderly patients with varying levels of cognitive and physical ability. All three patients have difficulty managing their own health data — one does not use a smartphone at all, another has early-stage dementia and cannot reliably remember to log readings, and the third is capable but resistant to technology. Jason currently manages access to all three patients' health portals by keeping their login credentials in a private notebook — a practice he knows is both a security risk and a HIPAA violation, but one he has no better alternative for.

What Jason needs from BioSync: a way to access patient health data without borrowing their credentials, a unified dashboard that shows him all three patients in one place, and visibility into readings and alerts without needing to be physically present. The caregiver linking system — where each patient generates a unique code that Jason enters to gain scoped access — was designed with Jason's situation as the central use case. He gets exactly what he needs for his role. He cannot see unrelated data, modify records he shouldn't touch, or access patients who haven't explicitly authorized him.

Would Jason come back? If the linking process is simple and the dashboard gives him actionable information — recent readings, active alerts, any new documents — without requiring him to dig for it, yes. The goal is for his morning check-in across all three patients to take five minutes, not fifty.

Persona 3 — Micah Davis, Chronic Illness Patient

Micah Davis is a 38-year-old with a chronic autoimmune condition that requires regular visits to three different specialists — a rheumatologist, a cardiologist, and a primary care physician. Each specialist has their own patient portal, and none of them share records with each other automatically. Micah has been hospitalized twice in the past year; each time, the discharging hospital changed or adjusted his medications, and the other two specialists were not informed until Micah's next scheduled appointment weeks later.

Micah is frustrated, exhausted, and medically at risk because of fragmented information. He tracks what he can in a notes app on his phone, but he cannot extract that data into a format any of his doctors can actually use. He wants a tool that stores all his medical records in one place, keeps a current medication list that reflects what he's actually taking right now, and that he can pull up on his phone during any appointment to show a doctor his full picture.

What Micah needs from BioSync: the personal health hub with document upload, the living medication list with automatic updates from uploaded discharge summaries, and the ability to show trend data to a physician rather than relying on memory. BioSync's medication list feature — which updates automatically when a new visit summary is uploaded — was designed specifically to address the scenario Micah experiences every time he leaves a hospital.

Would Micah come back? If the medication list stays accurate with minimal manual maintenance and he can pull it up at any appointment without fumbling through folders, yes. The key metric for Micah is accuracy with low effort.

4b. Core Value Proposition

BioSync's core value is consolidation with appropriate scoping. Most health tracking solutions solve for one side of the patient-caregiver relationship or one category of health data, but not both and not comprehensively. Fitness apps track activity but not medical records. Patient portals hold records but are siloed to one provider. No current consumer tool securely accommodates the caregiver role with appropriate data scoping, and none maintain a unified health picture across uploaded documents, manually logged readings, and device-generated data simultaneously.

BioSync does all of this in a single platform. A patient uploads their discharge paperwork, logs a blood pressure reading, and knows their caregiver can see exactly what they need — their readings, their alerts, their documents — without having to share their password or navigate a system designed only for the patient themselves. A doctor at an appointment can be shown weeks of trend data on a patient's phone rather than a single reading taken under the clinical stress of an office visit.

The thing BioSync does better than the alternatives — including doing nothing — is closing the communication gap between patients, caregivers, and providers. Every feature in the application was designed to reduce the friction of that communication: the unique patient code system, the living medication list, the role-based dashboards, the document upload and storage. BioSync is not trying to replace clinical systems. It is trying to be the connective tissue that those systems currently lack.

4c. Feature Scope

Status definitions — Implemented: working in the current build. Partial: started but not complete. Planned: designed but not built.

1. Biometric data logging — heart rate, blood pressure, blood oxygen, and additional metric types **[Implemented]**
2. Role-based user accounts — separate Patient and Caregiver flows from registration onward **[Implemented]**
3. Caregiver-to-patient linking via unique patient codes — explicit, patient-initiated authorization **[Implemented]**
4. Health alert generation — threshold-based notifications for out-of-range biometric readings **[Implemented]**
5. Medical document upload and server-side storage — visit summaries, discharge records, lab results **[Implemented]**
6. Medication list management — manual entry and update of current medications, doses, and status **[Implemented]**

7. Health data import/export — structured export to portable formats for sharing with providers **[Partial]**
8. Medication list auto-update from uploaded documents — extracting medication changes from visit summaries **[Partial]**
9. Data-driven health recommendations — pattern analysis producing proactive guidance based on trends **[Partial]**
10. Bluetooth and API device connectivity — direct reading ingestion from home monitors **[Planned]**
11. Native iOS and Android mobile applications **[Planned]**
12. Integration with third-party health apps (Apple Health, Google Fit, Fitbit) **[Planned]**

If the feature list were reduced to three must-have items — the minimum viable version of BioSync — the three survivors would be: biometric data logging across multiple metric types, caregiver-to-patient linking with role-scoped access, and medical document upload with medication list management. These three represent the application's core differentiation and the features that most directly address the pain points of all three personas described above.

5. User Stories

User stories are listed below in priority order, grouped by feature area. All stories follow the standard format: As a [user type], I want to [action] so that [benefit]. Technical implementation tasks are documented separately in the architecture and sprint summary sections and are not included here.

Story status reflects the state of each story as of the final Sprint 3 review. Stories marked Done were accepted by the Product Owner following demonstration of the working feature in the application.

Authentication & Account Setup

- ☐ As a new user, I want to create an account with a unique username and password so that I can access my personal health dashboard securely. **[Done — Sprint 1]**
  - ⊄ Acceptance: User can register, is redirected to login, and can successfully authenticate with new credentials.
- ☐ As a registered user, I want to log in to the application securely so that my health data is protected and accessible only to me. **[Done — Sprint 1]**
  - ⊄ Acceptance: Valid credentials create a signed session; invalid credentials return an error without revealing which field was wrong.
- ☐ As a user, I want my session to end when I log out so that my health data is not accessible if I leave the browser unattended. **[Done — Sprint 1]**
  - ⊄ Acceptance: Clicking logout immediately clears the session; navigating back does not restore access.

Role Selection & Profile Management

- ☐ As a new user, I want to select whether I am a Patient or a Caregiver during registration so that I am directed to the appropriate dashboard and features. **[Done — Sprint 2]**
  - ⊄ Acceptance: Registration form includes account type selection; selected type is persisted and determines dashboard routing.
- ☐ As a user, I want to view and update my profile information so that my account reflects accurate personal and health details. **[Done — Sprint 2]**
  - ⊄ Acceptance: Profile page displays current information and allows edits; changes are saved and reflected immediately.

Caregiver-Patient Linking

- ☐ As a patient, I want a unique code associated with my account so that I can share it with a caregiver to grant them access to my health data. **[Done — Sprint 2]**
  - ⊄ Acceptance: Each patient account has a visible unique code on their profile page. The code is generated at registration and persists.
- ☐ As a caregiver, I want to link to a patient by entering their unique code so that I can view their health data without needing their login credentials. **[Done — Sprint 2]**
  - ⊄ Acceptance: Caregiver enters a valid code; link is created; patient appears on caregiver dashboard. Invalid codes return an error.
- ☐ As a caregiver, I want to see all of my linked patients in one dashboard so that I can efficiently monitor multiple patients without switching accounts. **[Done — Sprint 2]**
  - ⊄ Acceptance: Caregiver dashboard lists all linked patients with most recent reading and any active alerts visible.

Biometric Data & Health Alerts

- ☐ As a patient, I want to log biometric readings including heart rate, blood pressure, and blood oxygen level so that I have a time-stamped record of my health over time. **[Done — Sprint 2]**
  - ⊄ Acceptance: User can select metric type, enter a value and unit, and submit; reading appears in their history with timestamp.
- ☐ As a patient, I want to receive an alert when a reading falls outside a normal range so that I know when to seek medical attention. **[Done — Sprint 2]**
  - ⊄ Acceptance: Readings outside defined thresholds trigger an alert visible on the dashboard. Alert includes the metric, value, and timestamp.

Document Management & Medication Tracking

- ☐ As a patient, I want to upload medical documents including visit summaries and discharge records so that I have a complete health history stored in one place. **[Done — Sprint 2]**
  - ⊄ Acceptance: User can upload a file; uploaded file appears in their document list; file is accessible to linked caregivers.
- ☐ As a patient, I want to maintain a medication list that I can update manually so that my current prescriptions are always reflected accurately. **[Done — Sprint 3]**
  - ⊄ Acceptance: Medication list shows current medications with name, dose, and status. User can add, edit, and mark medications as discontinued.
- ☐ As a patient, I want to export my health data and medication list so that I can share it with healthcare providers outside of the application. **[In Progress — Sprint 3]**
  - ⊄ Acceptance (target): User can download a PDF or structured data export of their biometric history and current medication list.

6. System Architecture

6a. Tech Stack

BioSync is built on a Python web stack with Flask at its core. Technology choices were primarily driven by course requirements and the team's existing familiarity, with secondary consideration for appropriate fit for the application's needs.

Technology	Category	Purpose and Rationale
Python 3.11	Language	Primary development language. Python's readability, strong standard library, and extensive web ecosystem made it the natural foundation. Course requirement for CSC 289.
Flask 3.x	Web Framework	Lightweight WSGI framework providing URL routing, request handling, session management, and Jinja2 templating integration. Chosen for simplicity and transparency over a heavier framework like Django.
Jinja2	Templating Engine	Server-side HTML templating system integrated with Flask. Enables role-based page rendering — the same base template can conditionally display patient or caregiver UI elements based on session data.
SQLite	Database	Lightweight relational database using Python's built-in sqlite3 module. Appropriate for development and low-concurrency deployment. A migration path to PostgreSQL is planned for any production deployment.
Werkzeug	Security / Utilities	Core Flask dependency providing password hashing via PBKDF2-SHA256, HTTP exception handling, and secure file upload utilities. Password hashing and session security are handled entirely through Werkzeug.
HTML5 / CSS3	Frontend	Standard markup and styling. Custom CSS follows the BioSync brand palette drawn from the website's style.css — navy (#1C2D4F), red (#E63329), and blue (#377BA8).
JavaScript (minimal)	Frontend Enhancement	Limited to form validation and basic UI interactions. BioSync is primarily a server-rendered application and does not use a JavaScript framework.
GitHub	Version Control	Collaborative source control. Team workflow uses feature branches with pull request review before merging to main.
GitHub Codespaces	Dev Environment	Cloud-based development environment pre-configured for the project. Ensures all team members develop against identical Python versions, dependencies, and database state.

6b. Application Structure

BioSync follows a standard Flask application structure. Routes are defined in app.py and map to Jinja2 templates in the templates/ directory. Business logic — threshold calculations, caregiver link validation, document handling — lives in route functions and extracted helper functions within app.py. The database is accessed directly via sqlite3 with parameterized queries; no ORM is used.

The application uses a single-file architecture with app.py as the entry point. For the scope of this project, this was appropriate. A production version would benefit from a Blueprint-based refactor to separate auth, patient, caregiver, and admin routes into distinct modules.

```
BioSync-Group2/   app.py                    Flask application: all routes,
helpers, and app config   schema.sql              Database schema — run once to
initialize tables   requirements.txt          Python dependencies (Flask,
Werkzeug, etc.)   README.md                Setup and deployment instructions
templates/     base.html               Master layout: nav bar, flash messages,
footer     index.html              Landing page     auth/       register.html
Registration form with role selection       login.html            Login form
dashboard/       patient.html          Patient dashboard: readings, alerts,
documents       caregiver.html        Caregiver dashboard: linked patients
overview     profile/       view.html             Profile display with patient
code       edit.html             Profile edit form     biometrics/       log.html
Reading entry form       history.html          Reading history with trend view
alerts.html           Alert list with severity indicators     medications/
list.html             Current medication list       edit.html
Add/update medication form     documents/       upload.html           Document
upload form       list.html             Uploaded document list    static/
style.css              Application stylesheet     images/                 Logo,
icons, and UI assets    uploads/                  Server-side document storage
(user-scoped paths)   instance/     biosync.db             SQLite database
(gitignored)
```

Application Request Flow

A typical authenticated request in BioSync follows this path:

13. User submits a form or navigates to a URL in the browser.
14. Flask's URL dispatcher matches the path to a route function in app.py.
15. The route function checks the session for an authenticated user ID and account type. If not present, the user is redirected to login.
16. If authenticated, the route queries the SQLite database using parameterized SQL. Role checks ensure that a caregiver cannot access patient data without an active link, and that a patient cannot access another patient's records.
17. Query results are passed to a Jinja2 template as context variables.
18. Jinja2 renders the template server-side, producing an HTML response which is returned to the browser.
19. For POST routes, form data is validated, the database is updated, and the user is redirected (POST-Redirect-GET pattern) to prevent duplicate submissions.

6c. Route Map

Route	Method	Description	Auth Required?
/	GET	Landing page. Authenticated users are redirected to their role-appropriate dashboard.	No
/register	GET, POST	New user registration. Collects username, password, confirm password, and account type (Patient or Caregiver). On success, redirects to login.	No
/login	GET, POST	User authentication. Validates credentials against hashed password; creates signed session on success.	No

Route	Method	Description	Auth Required?
/logout	GET	Clears the user session and redirects to the login page.	Yes
/dashboard	GET	Main dashboard. Renders patient or caregiver view based on session account type.	Yes
/profile	GET	User profile view. Displays personal information and, for patients, their unique linking code.	Yes
/profile/edit	GET, POST	Profile edit form. Allows updates to name, contact information, and health profile details.	Yes
/link	GET, POST	Caregiver-patient linking. Caregiver submits a patient code; a link record is created on success.	Yes (Caregiver)
/biometrics/log	GET, POST	Biometric reading entry. Patient selects metric type, enters value and unit, submits.	Yes (Patient)
/biometrics/history	GET	Reading history. Patient views their historical readings with timestamps and alert flags.	Yes
/alerts	GET	Alert list. Displays all triggered alerts with severity, metric, value, and timestamp.	Yes
/medications	GET	Medication list view. Shows current medications with name, dose, frequency, and status.	Yes
/medications/edit	GET, POST	Add or update a medication entry. Supports marking medications as discontinued.	Yes (Patient)
/documents/upload	GET, POST	Document upload. Patient uploads a file; stored server-side in user-scoped directory.	Yes (Patient)
/documents	GET	Document list. Shows all uploaded documents with name, date, and download link.	Yes

7. Data Model

7a. Entity Relationship Diagram

The diagram below represents BioSync's core database structure. Five primary tables handle user accounts, biometric readings, caregiver links, medications, and uploaded documents. The relationships are designed to enforce data scoping at the database level — a query for a patient's readings will only return results if the requesting user is the patient themselves or holds an active caregiver link.

[ERD Diagram — insert exported image from dbdiagram.io, draw.io, or equivalent tool]

Described textually: users is the central table. biometric_readings, medications, and documents all contain a user_id foreign key referencing users.id. caregiver_links is a join table with two foreign keys both referencing users.id — one for the caregiver and one for the patient. This structure enables the query pattern: 'give me all readings for patients where I have an active caregiver link' with a straightforward JOIN across caregiver_links and biometric_readings.

7b. Data Dictionary

`Table: users`

Central table for all application users. Both patient and caregiver accounts are stored here; the account_type field drives routing and access control throughout the application.

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	Unique user identifier. Auto-assigned at registration.
username	TEXT	UNIQUE, NOT NULL	User login name. Must be unique across all accounts. Case-sensitive.
password	TEXT	NOT NULL	PBKDF2-SHA256 hashed password with salt. Plain text is never stored.
account_type	TEXT	NOT NULL, CHECK	Either 'patient' or 'caregiver'. Set at registration and cannot be changed post-creation.
patient_code	TEXT	UNIQUE, NULLABLE	Randomly generated alphanumeric code for patient accounts. NULL for caregiver accounts. Used for caregiver linking.
first_name	TEXT	NULLABLE	User's first name. Set during profile setup.
last_name	TEXT	NULLABLE	User's last name. Set during profile setup.
date_of_birth	TEXT	NULLABLE	Patient date of birth in ISO 8601 format (YYYY-MM-DD). Used for health threshold context.
created_at	DATETIME	NOT NULL, DEFAULT	Timestamp of account creation. Defaults to current UTC time.

`Table: biometric_readings`

Stores all biometric readings logged by patients. One row per reading event. Designed to be queryable by metric type for trend analysis and by user+time for dashboard display.

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	Unique reading identifier.
user_id	INTEGER	FK → users.id, NOT NULL	Patient who recorded this reading. Foreign key with ON DELETE CASCADE.
metric_type	TEXT	NOT NULL	Type of biometric reading. Values: 'heart_rate', 'blood_pressure_systolic', 'blood_pressure_diastolic', 'blood_oxygen', 'temperature', 'weight'.
value	REAL	NOT NULL	Numeric value of the reading.
unit	TEXT	NOT NULL	Unit of measurement. Examples: 'bpm', 'mmHg', '%', '°F', 'kg'.
notes	TEXT	NULLABLE	Optional patient-entered context note for this reading (e.g., 'taken after exercise').
timestamp	DATETIME	NOT NULL	Date and time of the reading, stored in UTC. Displayed in user-local time on the frontend.
alert_triggered	INTEGER	NOT NULL, DEFAULT 0	Boolean flag (0 or 1). Set to 1 if this reading exceeded defined threshold ranges for its metric type.

Table: caregiver_links

Join table implementing the many-to-many relationship between caregivers and patients. Every row represents one active caregiver-patient relationship. The linking process is always initiated by the patient sharing their code with the caregiver — no row can exist in this table without the patient's active participation.

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	Unique link identifier.
caregiver_id	INTEGER	FK → users.id, NOT NULL	The caregiver account in this relationship.
patient_id	INTEGER	FK → users.id, NOT NULL	The patient account in this relationship.
linked_at	DATETIME	NOT NULL	UTC timestamp of when the caregiver successfully linked using the patient's code.
active	INTEGER	NOT NULL, DEFAULT 1	Boolean flag (0 or 1). Allows links to be deactivated without deleting the record, preserving audit history.

Table: medications

Stores each patient's medication list. Medications are never deleted — discontinued medications are marked with status='discontinued' and a discontinued_at timestamp, preserving a complete medication history for clinical reference.

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	Unique medication record identifier.
user_id	INTEGER	FK → users.id, NOT NULL	Patient account this medication belongs to.
name	TEXT	NOT NULL	Medication name as provided by the patient.
dose	TEXT	NULLABLE	Dosage information (e.g., '10mg', '2 tablets').
frequency	TEXT	NULLABLE	How often taken (e.g., 'once daily', 'twice daily with meals').
prescribing_provider	TEXT	NULLABLE	Name of prescribing physician or clinic, if known.
status	TEXT	NOT NULL, DEFAULT 'active'	Either 'active' or 'discontinued'. Active medications appear on the current list.
started_at	TEXT	NULLABLE	Date the medication was started, in ISO 8601 format.
discontinued_at	TEXT	NULLABLE	Date the medication was discontinued, if applicable.
notes	TEXT	NULLABLE	Additional notes such as reason for discontinuation or special instructions.

Table: documents

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	Unique document record identifier.
user_id	INTEGER	FK → users.id, NOT NULL	Patient account this document belongs to.
filename	TEXT	NOT NULL	Stored filename on the server (may be prefixed with user ID to prevent collisions).
original_name	TEXT	NOT NULL	Original filename as uploaded by the user. Displayed in the document list.
file_type	TEXT	NOT NULL	MIME type validated at upload (e.g., 'application/pdf', 'image/jpeg').
uploaded_at	DATETIME	NOT NULL	UTC timestamp of when the document was uploaded.
description	TEXT	NULLABLE	Optional user-provided description (e.g., 'Discharge summary from Wake Med 4/15/2026').

7c. Key Relationships

- users → biometric_readings (one-to-many): A single patient account has many biometric readings over time. Each reading stores the user_id as a foreign key. This enables the core time-series queries

that power both the patient dashboard (recent readings) and caregiver dashboard (patient readings via a link join).

- users → medications (one-to-many): A patient account has many medication records. The status field distinguishes active from discontinued medications, allowing the application to display a current list while preserving a complete medication history. This design decision was driven by the medication tracking use case — a discontinued medication should never disappear from the record.
- users → documents (one-to-many): A patient account has many uploaded documents. Documents are never shared across patients; access to a patient's documents by a caregiver is mediated entirely through the caregiver_links table.
- users → caregiver_links as caregiver (one-to-many): A single caregiver account can be linked to many patients, supporting professional caregivers who manage multiple patients from one account.
- users → caregiver_links as patient (one-to-many): A single patient can have multiple caregivers linked to their account — for example, both a home health aide and a family member.
- caregiver_links bridges caregivers and patients (many-to-many): Any query for caregiver-accessible data JOINs through this table, ensuring that caregivers can only reach data for patients with an active link record. The active flag allows soft-deletion of links without losing history.

8. Key Features & Implementation Notes

This section covers the four most significant features of BioSync in detail — from the user's perspective, the technical implementation approach, and the challenges encountered during development. These were the features that drove the most meaningful design decisions and produced the most learning.

Feature 1: Role-Based Authentication and Account Separation

What It Does

BioSync supports two completely distinct account types — Patient and Caregiver — selected once during registration and enforced throughout the application. A patient sees their own biometric dashboard, medication list, document storage, and alert history. A caregiver sees a list of linked patients with summary data for each. No route in the application is accessible to both types without conditional rendering — every protected endpoint checks both that a session exists and that the account type is appropriate.

The separation is not just cosmetic. Caregivers cannot navigate to patient data entry routes. Patients cannot navigate to caregiver linking or multi-patient management routes. This is enforced at the route level, not the template level — a caregiver who manually types in a patient-only URL will be redirected, not shown a hidden form.

How It Works

User credentials are stored in the users table with passwords hashed by Werkzeug's generate_password_hash() function using PBKDF2-SHA256 with a randomly generated salt. On login, check_password_hash() compares the submitted password against the stored hash — the original password is never recoverable. On successful authentication, Flask's session object (a cryptographically signed cookie using the application's SECRET_KEY) stores the user's ID and account type.

Every protected route begins with a session check. A decorator pattern would be the clean approach; in the current implementation, this check appears at the top of each route function. Routes that are type-specific additionally check the account_type value in the session. If either check fails, the user is redirected to login with an appropriate flash message.

Challenges Encountered

The primary challenge was consistency. Early in development, some routes had the session check and some did not — a common pattern when building features quickly across multiple contributors. The fix was a code review pass specifically targeting authentication gaps, followed by establishing a clear convention for how every new route should open. This also surfaced the need for the decorator-based approach as a future refactoring priority.

A secondary challenge was handling the edge case of a user who changes their account type after registration — which should not be allowed. The current implementation does not expose a UI mechanism to change account type, but does not enforce this constraint at the database level. A CHECK constraint on the users table and removal of any account_type update path in the route code would harden this before production.

Feature 2: Caregiver-Patient Linking via Unique Patient Code

What It Does

Every patient account in BioSync has a unique alphanumeric code displayed on their profile page. A caregiver who needs access to a patient's data receives this code from the patient — verbally, by text, however they choose to share it — and enters it into BioSync's caregiver linking form. Once submitted, a

caregiver_links record is created and the patient immediately appears on the caregiver's dashboard. The caregiver can then view that patient's biometric readings, medication list, alerts, and uploaded documents.

Critically, the caregiver sees a role-scoped view of each patient. They can view health data; they cannot edit the patient's profile, modify their medication entries, or delete their documents. The scope of caregiver access is intentionally narrow — exactly what is needed to support the patient's care, nothing more.

How It Works

Patient codes are generated using Python's secrets module — a cryptographically secure random string generator — at the time of account creation. The code is stored in the patient_code column of the users table with a UNIQUE constraint. The probability of collision is negligible at the code lengths used, but the UNIQUE constraint serves as a hard guarantee.

When a caregiver submits a code, the application queries users WHERE patient_code = [submitted_code] AND account_type = 'patient'. If a match is found and no active link already exists between this caregiver and this patient, a row is inserted into caregiver_links with both user IDs and a timestamp. Subsequent queries for caregiver-accessible patient data JOIN through this table: SELECT * FROM biometric_readings WHERE user_id = [patient_id] AND EXISTS (SELECT 1 FROM caregiver_links WHERE caregiver_id = [session_user_id] AND patient_id = biometric_readings.user_id AND active = 1).

Challenges Encountered

The most important design decision here was ensuring that the linking process is entirely patient-initiated and that caregivers have no discovery mechanism. An early version of the linking form allowed a caregiver to search by patient name — this was removed because it would allow a caregiver to find and attempt to link with any patient in the system, not just those who had shared their code. The current implementation requires possession of the patient's code as the only path to linking.

A secondary issue was duplicate link prevention. Without a check, a caregiver could submit the same code multiple times and create multiple link records, cluttering the caregiver_links table and creating ambiguity in queries. The current implementation checks for an existing active link before inserting a new one. A UNIQUE constraint on (caregiver_id, patient_id) in the caregiver_links table would provide a harder guarantee.

Feature 3: Biometric Data Logging and Health Alert System

What It Does

Patients can manually log biometric readings across multiple metric types: heart rate, systolic and diastolic blood pressure, blood oxygen percentage, body temperature, and weight. Each reading is timestamped and stored in the biometric_readings table. After each entry, the application evaluates the submitted value against a set of clinically-informed threshold ranges for that metric. If the value falls outside the normal range, an alert is generated and flagged on both the patient's dashboard and the dashboards of any linked caregivers.

The reading history view provides a time-ordered log of all entries with alert flags highlighted visually. The dashboard surfaces the most recent reading per metric type alongside any active alerts. Linked caregivers see the same summary for each of their patients.

How It Works

The threshold evaluation runs inside the route handler immediately after a new reading is saved. A dictionary maps each metric_type string to a tuple of (low_threshold, high_threshold). For example: THRESHOLDS = {'heart_rate': (60, 100), 'blood_oxygen': (95, 100), 'blood_pressure_systolic': (90, 140)}. The submitted value is compared to this range; if it falls outside, the reading's alert_triggered column is set to 1 before the row is committed.

Alert visibility is handled by a dashboard query that fetches readings WHERE alert_triggered = 1 AND timestamp > [window]. The alert display includes the metric type, value, unit, timestamp, and a note indicating whether the value was high or low relative to the threshold.

Challenges Encountered

Defining threshold values required careful research into clinical reference ranges. Heart rate in particular is context-dependent — a resting rate of 55 bpm is healthy for a fit adult but potentially concerning in a sedentary elderly patient. The current implementation uses conservative population-level thresholds as a starting point. A more sophisticated version would allow patients to set their own baseline ranges, or would factor in age and activity level from the profile. This is identified as a priority enhancement in the planned feature set.

Blood pressure presented a specific modeling challenge: it is two numbers (systolic and diastolic) that are conventionally presented together but need to be stored and evaluated independently. The current implementation logs each component as a separate biometric_readings row with metric_type = 'blood_pressure_systolic' or 'blood_pressure_diastolic', and the UI reconstructs the conventional X/Y display. This works but requires careful query construction to avoid presenting mismatched systolic/diastolic values from different readings.

Feature 4: Document Upload, Medication Tracking, and Secure Storage

What It Does

Patients can upload medical documents — visit summaries, discharge paperwork, lab results, imaging reports — directly to their BioSync profile. Uploaded documents are stored on the server in a user-scoped directory, accessible to the patient and their linked caregivers but no one else. Documents are never transmitted to third-party services; all processing and storage happens locally on the application server.

Separately, patients maintain a medication list within the application. Each medication entry includes name, dose, frequency, prescribing provider, start date, and status (active or discontinued). Discontinued medications are never deleted — they are marked with a status change and a discontinuation date, preserving a complete medication history. When a patient uploads a new visit summary or discharge document, the medication list can be updated to reflect changes noted in the document — new prescriptions, dose adjustments, or discontinued medications.

How It Works

File uploads use Flask's request.files interface with Werkzeug's secure_filename() utility to sanitize the original filename before storage. Files are saved to a uploads/{user_id}/ directory path, which means no user can construct a URL to another user's file even if they know the filename. The documents table stores the original filename, the stored filename, MIME type, upload timestamp, and an optional user-provided description. Flask's send_from_directory() serves files for download with the user_id path as a scope check.

The medication table stores all entries regardless of status. The active medication list query is: SELECT * FROM medications WHERE user_id = [patient_id] AND status = 'active' ORDER BY name. Discontinuing a medication updates the status to 'discontinued' and sets discontinued_at to the current timestamp — the row is never deleted. A separate query for medication history retrieves all rows including discontinued, ordered by started_at.

Challenges Encountered

File type validation proved more nuanced than expected. Browser-submitted MIME types can be spoofed — a malicious file can be given a .pdf extension and a PDF MIME type while containing executable content. The current implementation validates MIME type on the server using Python's mimetypes module as a second

check. For a production deployment, a dedicated file scanning library (such as python-magic for deep MIME inspection) would be used alongside virus scanning. This is flagged as a pre-launch security requirement.

The decision to use soft deletion for medications — marking as discontinued rather than deleting — emerged from a real use case. If a patient uploads a discharge summary showing that a medication was stopped, and then later the same medication is restarted, the history of both the original course and the new course is clinically significant. Hard-deleting the original row would lose that context. The soft-delete pattern also makes it possible to display a complete medication timeline, which is something the application's future recommendation engine will use.

9. Security & HIPAA Compliance

Overview

BioSync handles personal health information — biometric readings, medication lists, medical documents, and provider relationships. This is sensitive data by any standard, and it is protected health information (PHI) under HIPAA when handled in a healthcare context. Security and appropriate data handling were treated as first-class requirements throughout development, not as features to be added after the core functionality was complete.

The measures described below are implemented in the current build. A separate section describes what would be required before BioSync could be deployed in a real healthcare environment with real patient data.

Implemented Security Measures

Password Security

All user passwords are hashed using PBKDF2-SHA256 with a randomly generated per-user salt via Werkzeug's security module. This means that even if the database were compromised, the attacker would obtain only salted hashes — computationally expensive to crack and useless for cross-platform attacks (since each hash is unique to BioSync). Plain text passwords are never stored or logged at any point. Login authentication uses check_password_hash() to compare the submitted password against the stored hash, which performs the comparison in constant time to prevent timing attacks.

Session Management

User sessions are managed via Flask's built-in session object, which stores session data in a cryptographically signed cookie using the application's SECRET_KEY. The signing uses HMAC with SHA-1, ensuring that any modification to the session cookie will invalidate it — a client cannot forge or alter their session data without knowing the secret key. Sessions are invalidated and cleared on logout. The SECRET_KEY is stored as an environment variable and is never committed to the repository.

Data in Transit

The application is designed for HTTPS deployment. All communication between the client browser and the Flask server should be encrypted via TLS, ensuring that biometric readings, document uploads, medication data, and authentication credentials cannot be intercepted in transit. In the current development environment running over HTTP (Codespaces local), this protection is not active; it must be enforced before any real patient data is processed.

Database Access Control and Data Scoping

BioSync enforces data scoping at the application level through parameterized queries that always include the authenticated user's ID as a constraint. A patient can only retrieve their own biometric readings, medications, and documents. A caregiver can only retrieve data for patients where an active caregiver_links row exists with their user ID. No patient data is directly browsable — all queries are scoped to the requesting user's authorization context.

Parameterized queries are used throughout the application to prevent SQL injection. User-supplied values are never interpolated directly into SQL strings; they are always passed as parameters to sqlite3's execute() method.

Explicit Patient-Initiated Access Control

No caregiver can access any patient's data without a row in caregiver_links for that specific caregiver-patient pair, and no such row can be created without the patient sharing their unique code. There is no mechanism

in the application for a caregiver to discover or browse patient accounts. The unique code is the only pathway to access, and it is generated and shared by the patient alone.

Document Security

Uploaded medical documents are stored on the server in user-scoped directory paths (uploads/{user_id}/). Files are served through Flask routes that validate the requesting user's authorization before returning file contents — documents cannot be accessed by constructing a direct URL. Werkzeug's secure_filename() sanitizes all uploaded filenames before storage to prevent path traversal attacks. MIME type validation limits uploads to expected document types.

HIPAA Design Alignment

BioSync was designed with the core principles of HIPAA's Privacy Rule and Security Rule in mind. The following provisions are addressed by the current implementation:

HIPAA Principle	BioSync Implementation
Minimum Necessary Access	Caregivers receive a role-scoped view of patient data — only the information required to support their caregiving role. No caregiver has access to a patient's full account, login credentials, or administrative data.
Access Controls	Two-tier user system with strict route-level enforcement. Each account type is restricted to its own routes and data. Caregiver access requires explicit, patient-initiated authorization.
Audit Controls	Health alert notifications create a log of significant data events. Caregiver link records include timestamps of when access was granted. Biometric readings include timestamps for all entries.
Transmission Security	Designed for HTTPS/TLS deployment. No PHI is transmitted in plain text in the production configuration.
Integrity Controls	Session data is cryptographically signed to prevent tampering. Parameterized queries prevent injection attacks. Soft-deletion of medications and inactive caregiver links preserves data integrity.
No Third-Party PHI Transmission	Uploaded documents are processed and stored locally. Biometric data is not transmitted to any external service. No analytics or advertising integrations receive PHI.

Pre-Launch Security Requirements

The following measures would be required before BioSync could handle real patient data in a production healthcare environment. These represent the gap between a well-designed class project and a HIPAA-compliant production application.

20. Database encryption at rest — Migrate from SQLite to PostgreSQL with full encryption at rest for all PHI, biometric records, and uploaded documents. SQLite does not natively support encryption at rest.
21. Full HTTPS enforcement — SSL/TLS certificates on all production routes with HTTP Strict Transport Security (HSTS) headers to prevent protocol downgrade attacks.
22. Third-party security audit and penetration testing — Independent assessment before any patient data goes live. No in-house security review is a substitute for external penetration testing.

23. Comprehensive audit logging — Every data access event, login attempt, failed authentication, caregiver link creation, document view, and record modification logged with user ID, timestamp, and IP address. HIPAA requires audit trails.
24. Formal HIPAA Business Associate Agreements — BAAs required with all third-party vendors in the data path. Data flow documentation and PHI handling procedures must be formalized.
25. Token-based authentication — Replace Flask session cookies with JWT or OAuth 2.0 for stateless, scalable, and more auditable authentication. Essential for future mobile app support.
26. Rate limiting and brute-force protection — Lock out repeated failed login attempts per IP and per account. CAPTCHA on authentication endpoints. Prevents credential stuffing attacks.
27. Automated encrypted backups with disaster recovery procedures — Regular encrypted backups with tested restore procedures and documented RTO/RPO targets before any production deployment.
28. File scanning for uploaded documents — Server-side virus scanning and deep MIME type inspection (using python-magic or equivalent) for all uploaded files. Prevents malicious file uploads.
29. Patient consent and data use documentation — Explicit informed consent flow for patients, privacy policy disclosure, and data retention/deletion policy before any real patient data is accepted.

10. Deployment

Current Deployment Environment

BioSync is currently developed and run within GitHub Codespaces — a cloud-based development environment that provides each team member with a pre-configured, identical runtime environment. Codespaces provisions a Linux container with Python, Flask, and all dependencies pre-installed based on the repository's devcontainer configuration. The application runs on Flask's built-in development server (flask run) within this environment, accessible via a port-forwarded URL.

This setup is appropriate for development and demonstration but is not production-ready. Flask's development server is single-threaded, not designed for concurrent load, and includes a debugger that must never be exposed in production. A production deployment requires several additional steps described below.

Production Deployment Path

For a production deployment of BioSync, the recommended platform is Render, Railway, or PythonAnywhere — all of which support Python/Flask applications with minimal configuration and provide managed SSL/TLS certificates. The following changes are required to move from the current development setup to a production deployment:

- Replace SQLite with PostgreSQL — SQLite is not suitable for concurrent production traffic. The psycopg2 library and a DATABASE_URL environment variable allow Flask to connect to a managed PostgreSQL instance on Render or Railway with minimal code changes.
- Replace Flask dev server with Gunicorn — Gunicorn is a production-grade WSGI server that handles concurrent requests and is standard for Flask deployments. Add gunicorn to requirements.txt and update the start command to: gunicorn app:app --bind 0.0.0.0:$PORT
- Enforce HTTPS — The hosting platform handles TLS termination; the Flask application must set SESSION_COOKIE_SECURE = True and SESSION_COOKIE_HTTPONLY = True in the production config to ensure session cookies are only transmitted over HTTPS.
- Set environment variables — All secrets must be stored as environment variables in the hosting platform's dashboard, never in the codebase.
- Configure file storage — The uploads/ directory either needs to be on persistent storage (most platforms provide this) or migrated to object storage (AWS S3, Cloudflare R2) for scalability and durability.

Required Environment Variables

Variable	Purpose and Notes
SECRET_KEY	Flask session signing key. Must be a long, randomly generated string in production. Generate with: python -c "import secrets; print(secrets.token_hex(32))"
DATABASE_URL	Database connection string. SQLite path in development (sqlite:///instance/biosync.db). PostgreSQL URL in production (postgresql://user:password@host/dbname).
UPLOAD_FOLDER	Server-side path for storing uploaded patient documents. Must be writable by the application process. Example: /var/app/uploads

Variable	Purpose and Notes
MAX_CONTENT_LENGTH	Maximum allowed file upload size in bytes. Recommended: 16777216 (16MB). Larger uploads should be rejected at the application level.
FLASK_ENV	Set to 'production' in deployment to disable the debugger and enable production error handling. Never set to 'development' in production.
ALLOWED_EXTENSIONS	Comma-separated list of allowed file upload extensions. Example: pdf,jpg,jpeg,png. Validated at upload time.

Step-by-Step Deployment Instructions

The following steps assume deployment to Render using the free tier. Adjust service names and URLs for other platforms.

30. Fork or clone the repository: git clone https://github.com/CSC-289-Biometric-Group-2/BioSync-Group2
31. Ensure requirements.txt includes: flask, werkzeug, gunicorn, and psycopg2-binary (for PostgreSQL).
32. Create a Procfile in the project root with the content: web: gunicorn app:app
33. Create a new Web Service on Render, connect the GitHub repository, and set the build command to: pip install -r requirements.txt
34. Add all environment variables listed above in the Render dashboard under Environment.
35. Provision a PostgreSQL database on Render and copy the internal DATABASE_URL into the web service's environment variables.
36. On first deploy, initialize the database schema: flask --app app init-db (or run the SQL in schema.sql against the production database directly).
37. Verify the deployment at the provided Render URL. Check that HTTPS is active and that the application redirects HTTP to HTTPS.

Known Gotchas and Deployment Notes

- SQLite database file — The instance/biosync.db file must not be committed to the repository. It is in .gitignore, but verify this before pushing to a public repo. On production, the database connection should use the DATABASE_URL environment variable.
- uploads/ directory — This directory must exist before the application starts. On Render, persistent disk storage must be explicitly provisioned and mounted. Without this, uploaded documents will be lost on every redeploy.
- SESSION_COOKIE_SECURE — This must be True in production (HTTPS) and False in development (HTTP). Setting it to True in a development environment without HTTPS will break all sessions. Use a config class to manage this conditionally.
- Debug mode — Ensure FLASK_DEBUG or FLASK_ENV is never set to a development value in production. The Flask debugger exposes a remote code execution interface when enabled.
- File permissions — The uploads/ directory must be writable by the user account running the Gunicorn process. On containerized platforms, this is usually handled automatically, but may require explicit chmod in a Dockerfile.
- Codespaces port forwarding — When demoing from Codespaces, the forwarded port URL changes each time the environment is restarted. Update any hardcoded URLs in templates or configuration if this causes issues.

11. Lessons Learned & Retrospective

This section contains individual reflections from each Group 2 team member, covering what they learned, what they would do differently, and what they are most proud of from this project. Reflections are written in the first person and represent each member's genuine perspective on the work.

> *Note to team: The paragraphs below are placeholders. Please replace each one with your own reflection before final submission. Your reflection should be at least two substantial paragraphs — one covering learning and challenges, one covering what you would do differently and what you are proud of.*

Sarah Ayres — Product Owner, Scrum Master, Frontend Lead

This project taught me more about the gap between building something and building something right than any course work before it. Early on, I found myself moving fast — getting features to work, getting demos ready, keeping momentum up. What I underestimated was how much time the second pass takes: the security review, the access control audit, the documentation, the edge cases that only surface when you actually think through what a real user would do. The parts of BioSync I am most proud of are the ones that came from that second pass — the caregiver linking system, the session management, the file scoping — because those are the parts that took the most deliberate thought and would hold up in a real environment.

[Complete this reflection with: what specific skill or tool you learned that you didn't have before, what you would do differently with the benefit of hindsight, and the moment or contribution from this project you are most proud of.]

Joshua Eckman — Developer

Working on the application initialization gave me a deeper understanding of how Flask routes, context, and the request lifecycle actually work — not just as documentation concepts but as living code I had to debug and reason about. Setting up the database schema and the initial routing structure forced me to think about the whole application before any of it was built, which is a different kind of thinking than writing a single feature. The heart rate threshold work was satisfying specifically because it connected the technical (threshold comparison, alert flagging) to something real — the difference between a reading that's fine and one that needs attention.

[Complete this reflection with: what you would do differently, particularly around any technical decisions that created work for yourself or teammates later, and what you are most proud of from your contributions to the project.]

Myra Williams — Scrum Master, Developer

[Complete this reflection — address: a specific skill or concept you learned during this project that you did not have before (this could be technical, process-related, or collaborative), what you would do differently if you were starting the project over today with the knowledge you have now, and one specific contribution or moment from this project that you are proud of. Your reflection should be two substantial paragraphs.]

Toni Wilson — Developer

[Complete this reflection — address: a specific skill or concept you learned during this project that you did not have before (this could be technical, process-related, or collaborative), what you would do differently if you were starting the project over today with the knowledge you have now, and one specific contribution or moment from this project that you are proud of. Your reflection should be two substantial paragraphs.]

Team Retrospective

Looking at the project as a whole, Group 2 built a genuinely functional, thoughtfully designed health platform from scratch in a single semester. The caregiver linking system is something none of us had seen in existing tools. The security implementation is more considered than most student projects. And the application addresses a real problem — one that one of our team members lives with every day.

If we were doing this again, the things we would prioritize differently are: getting a consistent authentication decorator established in Sprint 1 rather than retrofitting it, agreeing on a database naming convention before anyone wrote their first query, and scheduling dedicated code review sessions rather than leaving review to happen ad hoc in pull requests. These are small process improvements that would have saved time and reduced integration friction across sprints.

What we are proudest of is the coherence of the final product. BioSync does not feel like four people's features bolted together — it feels like one application with a consistent design language, a consistent security posture, and a consistent user experience. That coherence came from clear product ownership, a real understanding of the problem we were solving, and a team that took the work seriously.

12. Appendices

The following appendices supplement the main binder content with reference materials, design assets, and supporting documentation. Appendices do not count toward the 20–30 page target.

Appendix A — Wireframes and UI Mockups

BioSync's visual design was informed by the VitalWatch concept developed by GD students at Wake Tech. The application's color palette, typographic approach, and component style (card-based layouts, role-selection buttons, dashboard structure) were adapted from that brand system into Flask/HTML/CSS templates.

Key UI screens in the current build include:

- Landing page — BioSync logo, brief description, and login/register call to action buttons styled in the brand's navy (#1C2D4F) and red (#E63329)
- Registration — clean form with a role selection step using the caregiver/individual button pattern from the original VitalWatch UI
- Patient dashboard — metric summary cards, recent readings list, active alert callouts, quick-access document list
- Caregiver dashboard — linked patient list with per-patient reading summary and alert status
- Biometric log — metric type selector, value and unit inputs, notes field, submission confirmation
- Medication list — active medications table with add/edit/discontinue actions

[Insert screenshots of the implemented application UI here. Before/after wireframe comparisons are also appropriate if early design artifacts are available.]

Appendix B — Pitch Presentation

Group 2 delivered a class presentation of BioSync including a live demo of the application. The presentation was built in PowerPoint using the BioSync brand palette and included an introduction from each team member, a problem statement grounded in personal caregiving experience, a feature walkthrough, a security and HIPAA compliance section, a pre-launch roadmap, and a future features section covering native mobile apps, Bluetooth device connectivity, and health app integration.

- Presentation file: BioSync_v3.pptx (available in the project repository)
- Q cards: BioSync_QCards.pdf (presenter reference cards with scripted notes and tips for each slide)
- Presentation structure: 10 slides plus a Starting Soon hold screen and a closing slide with the personal note about continuing development

Appendix C — API Documentation

BioSync does not currently expose a public REST API. All data access is through server-rendered HTML routes. The route table in Section 6c documents all URL endpoints.

For future development — particularly native mobile app support — a JSON API layer would be added. The planned endpoint structure would follow REST conventions:

Planned Endpoint	Method	Description (Future)
/api/readings	GET, POST	Retrieve or submit biometric readings for the authenticated patient.
/api/readings/{id}	GET, DELETE	Retrieve or delete a specific reading by ID.

Planned Endpoint	Method	Description (Future)
/api/medications	GET, POST	Retrieve current medication list or add a new medication.
/api/medications/{id}	PUT, PATCH	Update a medication record (e.g., mark as discontinued).
/api/patients	GET	Caregiver-only: list all linked patients with summary data.
/api/patients/{id}/readings	GET	Caregiver-only: retrieve readings for a specific linked patient.
/api/alerts	GET	Retrieve all active alerts for the authenticated user.

Appendix D — Meeting Notes and Sprint Summaries

BioSync was developed across three two-week sprints. Key decisions and outcomes from each sprint are summarized below. Full standup logs and sprint notes are maintained in the GitHub repository.

Sprint 1 Summary

Sprint 1 focused on project scaffolding and foundational authentication. The team established the GitHub repository, configured the Codespaces environment, designed the initial database schema, and implemented user registration, login, logout, and basic session management. The primary technical challenge was agreeing on the database schema before development began — in particular, the decision to store both patients and caregivers in a single users table with an account_type discriminator rather than separate tables. This decision simplified the authentication code but required more careful access control implementation throughout.

- Delivered: User registration, login/logout, session management, basic Flask routing structure
- Not completed: Role-based dashboard routing (pushed to Sprint 2 after scope clarification)
- Key decision: Single users table with account_type discriminator

Sprint 2 Summary

Sprint 2 was the most feature-intensive sprint, delivering role selection, the caregiver linking system, profile management, biometric data logging, health alert generation, and document upload. The caregiver linking system was the most complex feature of this sprint — both technically (the JOIN-based access control) and from a design perspective (ensuring that the linking process was entirely patient-initiated). Document upload also required more careful security consideration than anticipated.

- Delivered: Role-based dashboards, caregiver linking, profile management, biometric logging, health alerts, document upload
- Not completed: Medication list (pushed to Sprint 3)
- Key decision: Caregiver discovery removed from linking form — code-only access to prevent unauthorized linking attempts

Sprint 3 Summary

Sprint 3 focused on hardening, documentation, and completion of remaining features. The medication list was implemented with soft-deletion semantics. Security was reviewed across all routes and file handling code. The project binder, presentation, and Q cards were completed. Export functionality was started but not completed within the sprint. The team completed the demo preparation and presentation delivery.

- Delivered: Medication list with soft-deletion, security review, documentation, presentation, demo
- Not completed: Structured data export (in progress)
- Key decision: Soft-deletion pattern for medications to preserve clinical history

Appendix E — Design Assets and Brand Reference

The following design assets were used in or informed the BioSync project. All visual design decisions — colors, typography, layout patterns, component style — are traceable to these sources.

Asset	Source / File	Usage in BioSync
BioSync Logo	biosync.png (2000×2000 PNG)	Application icon, binder cover, header, all print materials
Navy color #1C2D4F	style.css (.btn-caregiver)	Primary nav color, heading backgrounds, table headers
Red color #E63329	style.css (.btn-individual)	Accent color, section rule underlines, 'Sync' in title
Blue color #377BA8	style.css (h1, a)	Subheadings, links, secondary accents
Light blue #CAE6F6	style.css (.flash)	Flash message backgrounds, subtle highlights
VitalWatch brand guide	GD student deliverable	Inspired color palette, caregiver/individual role split, heartbeat motif

Appendix F — Planned Feature Roadmap

The following features are planned for future development of BioSync beyond the capstone submission. They represent the full product vision — what BioSync would become if continued as a real product.

Feature	Priority	Description
Native iOS & Android Apps	High	Dedicated mobile applications providing the full BioSync experience in a native mobile form factor. Essential for adoption by patients who do not regularly use a desktop browser.
Bluetooth & API Device Sync	High	Direct integration with home monitoring devices — blood pressure cuffs, glucose meters, pulse oximeters — via Bluetooth or manufacturer APIs. Readings logged automatically without manual entry.
Health App Integration	High	Sync with Apple Health, Google Fit, and Fitbit to pull in steps, sleep, and activity data alongside clinical readings. Provides a complete health picture for appointments.
Medication Auto-Update from Documents	High	Automatically extract medication changes from uploaded visit summaries and discharge documents, updating the living medication list without manual entry.
Doctor-Ready Health Reports	Medium	One-tap generation of a PDF health summary including biometric trends, current medications, and recent alerts — formatted for presentation to a healthcare provider.
Cross-System File Sharing	Medium	Secure, direct file sharing between BioSync and external healthcare systems (VA portal, EHR systems) via HL7 FHIR or equivalent health data interoperability standards.
Pattern-Based Recommendations	Medium	AI-assisted analysis of biometric trends to generate proactive health guidance — e.g., 'Your blood pressure readings have

Feature	Priority	Description
		been consistently elevated over the past 30 days; consider discussing with your provider.'
Multi-language Support	Low	Interface localization for Spanish and additional languages to improve accessibility for non-English-speaking patients and caregivers.